Computers and Operations Research 81 (2017) 269–281

Contents lists available at ScienceDirect

Computers and Operations Research

journal homepage: www.elsevier.com/locate/cor

The probabilistic orienteering problem

Enrico Angelelli, Claudia Archetti, Carlo Filippi*, Michele Vindigni

Department of Economics and Management, University of Brescia, Contrada S. Chiara 50, 25122 Brescia, Italy

ARTICLE INFO

Article history:
Received 15 April 2016
Revised 22 December 2016
Accepted 28 December 2016
Available online 31 December 2016

Keywords:
Stochastic routing
Orienteering problem
Probabilistic TSP
L-shaped method
Matheuristics

ABSTRACT

The probabilistic orienteering problem (POP) is defined on a directed graph where a cost is associated with each arc and a prize is associated with each node. Moreover, each node will be available for visit only with a certain probability. A server starts from a fixed origin, has a given budget to visit a subset of nodes, and ends at a fixed destination. In a first stage, a node subset has to be selected and a corresponding a priori path has to be determined such that the server can visit all nodes in the subset and reach the destination without exceeding the budget. The list of available nodes in the subset is then revealed. In a second stage, the server follows the a priori path by skipping the absent nodes. The POP consists in determining a first-stage solution that maximizes the expected profit of the second-stage path, where the expected profit is the difference between the expected total prize and the expected total cost.

We discuss the relevance of the problem and formulate it as a linear integer stochastic problem. We develop a branch-and-cut approach for the POP and several matheuristic methods, corresponding to different strategies to reduce the search space of the exact method. Extensive computational tests on instances with up to 100 nodes show the effectiveness of the exact method and the efficiency of the matheuristics in finding high quality solutions in a few minutes. Moreover, we provide an extended analysis on a subset of instances to show the value of explicitly modeling the stochastic information in the problem formulation.

1. Introduction

Stochastic routing problems have received much attention in recent years. These are problems arising in the field of transportation and logistics for which the input is, at least partially, modelled in a stochastic way. Stochastic routing problems are considerably more difficult to solve than their deterministic counterparts. In fact, the notion of a solution is different and some properties that hold in a deterministic case are not valid in the stochastic context [12]. However, stochastic routing problems capture uncertainties that very often arise in real-world decisions. In particular, time-constrained deliveries are one of the fastest growing segments of the delivery business, but little effort has been done so far to address time constraints in the context of stochastic customer presence [10]. Moreover, it is argued that selective vehicle routing problems are more appropriate than the conventional routing problems in handling uncertainty with limited resources [1].

We consider a time-constrained, selective, stochastic routing problem. This is a stochastic version of the orienteering problem, a well-known deterministic selective routing problem that embeds characteristics of the travelling salesman problem (TSP) and the knapsack problem (KP). Consider a graph where nodes represent customers; a service courier, located at a distinguished origin, visits a subset of customers and ends to a distinguished destination, generally different from the origin. A prize is collected for every visited customer, a time is needed for every traversed arc, and a maximum total time is given. The orienteering problem (OP) consists in finding a path from the origin to the destination that maximizes the sum of collected prizes while respecting the maximum time constraint. The OP is introduced by Tsiligirides [31] and is shown to be NP-hard by Golden et al. [20]. The variant of the OP where a tour is sought instead of a path is usually known as selective TSP. A survey on the OP is given by Vansteenwegen et al. [32].

In our problem, each customer requires service only with a specified probability, and the sequence of decisions and information is as follows. In the first stage, an a priori path from the origin to the destination, covering a subset of customers within the specified total time has to be designed. In the second stage, information about the customers to be visited is available, and the customers are visited in the same order as they appear in the a priori path by simply skipping the absent ones. The profit of the a priori path is defined as the difference between the sum of collected prizes among the visited customers and the cost of travelling the

* Corresponding author.
E-mail addresses: enrico.angelelli@unibs.it (E. Angelelli), claudia.archetti@unibs.it (C. Archetti), carlo.filippi@unibs.it (C. Filippi), michele.vindigni@unibs.it (M. Vindigni).

http://dx.doi.org/10.1016/j.cor.2016.12.025
0305-0548/

path skipping the absent customers, where we assume that time and cost are proportional and satisfy the triangle inequality. Notice that the actual profit is revealed only after the design of the a priori path. The probabilistic orienteering problem (POP) consists in finding an a priori path of maximum expected profit with respect to all possible subsets of nodes to be visited and such that the total time of the path cannot exceed the maximum time limit.

The POP is both a novel stochastic version of the OP and an extension of the probabilistic travelling salesman problem [23]. It is then worth of attention in its own.

Nevertheless, the original motivation for this work comes from a problem faced by a transport company located in northern Italy, operating in the less-than-truckload (LTL) industry, carrying goods from northern Italy to northern Europe and vice versa. The company faces a complex pick-up and delivery problem. During a truck journey, a truck driver starts from the firm's headquarters, collects parcels from different customers directed to a certain area, e.g., The Netherlands, makes his journey to the destination area, and delivers the parcels. The process is then repeated with different origin and destination areas. Since most parcels are relatively small in size and weight, truck capacity is not binding. However, due to legal issues (Hours of Service regulations), organizational issues (compliance with a preliminary schedule), and convenience issues (overnight at partner hotels), the duration of the trip, and in particular the duration of the collection phase and the duration of the delivery phase, is constrained.

Customers requiring a transportation service have different levels of trust and reliability. They range from frequent, well-known customers, to spot customers met on an e-marketplace. Requests from well-known customers may be considered as mandatory, but requests from spot customers depend on a negotiation phase whose result is in general uncertain. Based on its experience and other factors, the firm is able to a priori evaluate the reliability of each spot customer and thus the probability that a certain shipping will be requested from a certain location.

Every time a new trip is planned, the firm has to include the mandatory customers and to select a subset of possible transportation requests from spot customers to negotiate. The firm wants to be able to satisfy all the selected requests, though it is not guaranteed that all of them will be eventually arranged. Notice that the revenue associated with a certain shipping is quite certain, since it follows standard pricing rules. However, the possibility that such a shipping will be arranged can be highly uncertain.

The firm needs a tool to decide which spot customers should be selected for the negotiation phase. This tool requires a quick evaluation of the expected revenue and the expected traveling cost and time associated with any customer subset. While the expected revenue is easy to obtain, an exact evaluation of the expected cost is intractable. Hence, we may resort to a priori optimization as a reasonable evaluation approach. In practice, once the subset of customers requiring service is revealed, an exact TSP computation could be performed on the remaining selected customers.

In summary, the POP models a simplified form of each pick-up phase, where a static approximation of a dynamic decision problem is used, but some crucial aspects of the real problem are retained. In fact, studying the POP may be seen as a first step in the analysis of the real problem.

In this paper, we define the POP and formulate it as a stochastic mixed integer linear program (MILP), deriving explicitly a deterministic equivalent. We then design an exact branch-and-cut method, discussing in particular specific branching rules based on the concept of chain. Moreover, we propose different strategies to reduce heuristically the search space of the branch-and-cut method, using heuristic branching and variable fixing. All algorithms have been implemented and tested, showing the effectiveness of the exact method and the efficiency of the heuristic strategies in finding high quality solutions in a few minutes.

The main contributions of this paper can be summarized as follows:

- We introduce a new problem, namely the Probabilistic Orienteering Problem (POP), that generalizes the well-known Orienteering Problem by introducing stochastic service requests.
- We propose a mathematical formulation for the POP.
- We develop a branch-and-cut algorithm for the POP where we introduce ad-hoc branching rules which prove to be effective in speeding up the solution process.
- We develop a matheuristic algorithm which is based on the branch-and-cut scheme and introduces smart heuristic rules to reduce the solution space and get feasible solutions fast.
- We perform an extensive analysis to show when the application of the POP is valuable, i.e., when it is worthwhile to explicitly model the stochastic information as done in the POP.

The most innovative contributions, in terms of solution methodologies, are given by the branching rules, which prove to be crucial to succeed in solving reasonable size instances, and the matheuristic algorithm, which is based on the idea of exploiting the information collected during the exploration of the branch-and-bound tree to cut-off unpromising branches and fix variables values.

The paper is organized as follows. In Section 2, we define the POP; in Section 3, we review the most relevant literature; in Section 4, we formulate the MILP model. In Section 5, the branch-and-cut is described, while the matheuristics are briefly discussed in Section 6. Section 7 shows the computational results. In Section 8, we analyse the impact of stochastic information on the solution of the POP and, in Section 9, we give some conclusions.

2. Problem definition

In this section, we formally define the POP and we discuss the specific objective function we choose. We assume that there are no mandatory customers, i.e., every customer may be selected or not. This allows a neater definition and simplifies the comparison with the classic orienteering problem and its variants. The role of mandatory customers will be discussed in Section 8.

Let $V' = \{1, 2, \ldots, n\}$ be the set of n potential customers, let 0 be the origin and let $n+1$ be the destination. In order to serve a subset of customers, a server starts from the origin, visits the customers according to some order, and then ends at the destination. Let $V = V' \cup \{0, n+1\}$. Whenever a customer $i \in V'$ is visited and served, a prize p_i is collected. The availability of a customer i for service is random and is modelled by a Bernoulli variable $b_i = \{0, 1\}$ that takes value 1 (availability for service) with probability π_i. Since origin and destination must be visited and do not provide any prize, we may set $\pi_0 = \pi_{n+1} = 1$ and $p_0 = p_{n+1} = 0$. We assume that the customers behave independently.

Let $G = (V, A)$ be a directed graph where $A = \{(i, j) : i \in V' \cup \{0\}, j \in V' \cup \{n+1\}\}$. Let t_{ij} be the travelling time assigned to arc $(i, j) \in A$. Consider an elementary path starting in 0 and ending in $n+1$:

$$\mathcal{P} : i_0 = 0, i_1, i_2, \ldots, i_q, i_{q+1} = n+1.$$

If the path is traversed thoroughly, the total collected prize is

$$P_{\max}(\mathcal{P}) = \sum_{k=1}^{q} p_{i_k}.$$

However, we are not guaranteed that all customers along the path will be available for service. Thus, the actual prize of path $\mathcal{P}$ is

$$P(\mathcal{P}) = \sum_{k=1}^{q} b_{i_k} p_{i_k},$$

and the expected total prize is

$$\mathbb{E}[P(\mathcal{P})] = \sum_{k=1}^{q} \pi_{i_k} p_{i_k}, \tag{1}$$

where $\mathbb{E}[\cdot]$ denotes the expectation operator.

Similarly, if the path is traversed thoroughly, it requires a total time

$$T_{\max}(\mathcal{P}) = \sum_{k=0}^{q} t_{i_k, i_{k+1}}. \tag{2}$$

A maximum duration $D_{\max}$ cannot be exceeded; hence we impose that $T_{\max}(\mathcal{P}) \leq D_{\max}$. If customer i_k is not available for service, its position is skipped along the path. It follows from Theorem 1 in [25] that the realized time of $\mathcal{P}$ is

$$T(\mathcal{P}) = \sum_{h=0}^{q} \left[b_{i_h} \sum_{k=h+1}^{q+1} \left(t_{i_h i_k} \cdot b_{i_k} \cdot \Pi_{j=h+1}^{k-1} (1 - b_{i_j}) \right) \right]. \tag{3}$$

By using the independence assumption, we may write

$$\mathbb{E}[T(\mathcal{P})] = \sum_{h=0}^{q} \left[\pi_{i_h} \sum_{k=h+1}^{q+1} \left(t_{i_h i_k} \cdot \pi_{i_k} \cdot \Pi_{j=h+1}^{k-1} (1 - \pi_{i_j}) \right) \right], \tag{4}$$

where $\pi_0 = \pi_{n+1} = 1$ and it is assumed that $\Pi_{j=h+1}^{k-1}(1 - \pi_{i_j}) = 1$ whenever $h + 1 > k - 1$. Note that, by construction, (4) implies a recourse function which establishes to skip those customers which will not appear once customers' information is provided. In fact, the expected cost of the a priori tour is calculated by adding the cost of the arcs joining pairs of customers visited consecutively in the sequence while the customers in the middle do not appear.

We assume that the cost of a path is proportional to the time it requires; let C denote the cost per time unit. The *probabilistic orienteering problem* (POP) consists in finding an elementary path $\mathcal{P}$ of maximum expected profit among those with total duration not greater than $D_{\max}$, where the expected profit is the difference between the expected total prize $\mathbb{E}[P(\mathcal{P})]$ and the expected total cost $C \cdot \mathbb{E}[T(\mathcal{P})]$.

The choice of the objective function deserves a brief discussion. In the deterministic orienteering problem (OP), there is no uncertainty about the availability of customers, and the objective is simply to maximize the total prize. Hence, a natural idea for the POP is to maximize the expected total prize $\mathbb{E}[P(\mathcal{P})]$ defined in (1). However, this variant of the problem would boil down to an OP where the prize associated with customer i is $p'_i = \pi_i p_i$ for all $i \in V'$. Another possibility is to maximize the difference between the expected total prize $\mathbb{E}[P(\mathcal{P})]$ and the maximum total cost $C \cdot T_{\max}(\mathcal{P})$. This would lead again to a completely deterministic model and would not consider the fact that the recourse action could save potential costs. Conversely, the proposed objective function $\mathbb{E}[P(\mathcal{P})] - C \cdot \mathbb{E}[T(\mathcal{P})]$ gives rise to a stochastic model where the recourse action is adequately considered.

To illustrate the difference in the three objectives, consider a simple example with four customers, where $p_i = 10$ and $\pi_i = 0.5$ for all $i = 1, 2, 3, 4$. Suppose that $C = 1$ and that the travel times follow a symmetric path metric, shown in Fig. 1, where the numbers below the nodes are i values, and the numbers above the edges are times. For instance, the path 0,3,1,5 has a total cost of 14. We let $D_{\max} = 18$.

In this example, there are only five non-dominated, non-trivial feasible paths. They are compared in Table 1 with respect to the different objective functions. We see that every objective gives a different optimal solution, corresponding to the maximum value in each column, which is highlighted in bold.

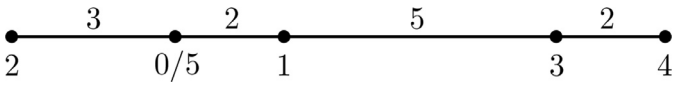

Fig. 1. A simple orienteering problem.

Table 1
Comparison of objectives on the example of Fig. 1.

$\mathcal{P}$	$\mathbb{E}[P(\mathcal{P})]$	$\mathbb{E}[P(\mathcal{P})] - C \cdot T_{\max}(\mathcal{P})$	$\mathbb{E}[P(\mathcal{P})] - C \cdot \mathbb{E}[T(\mathcal{P})]$
0, 1, 5	5	**1**	3
0, 2, 5	5	−1	2
0, 1, 2, 5	10	0	**5**
0, 1, 3, 5	10	−4	2
0, 1, 3, 4, 5	**15**	−3	2

Thus, we cannot bring back the POP to one of the simpler deterministic problems described above.

A further consideration related to the objective function is the following. In the classic OP, the objective function is the maximization of the expected total prize. In the Probabilistic Orienteering Problem, the objective function is the maximization of the difference between the expected total prize and the expected total cost. In this sense, the POP combines the objective function of the Profitable Tour Problem (PTP) with a maximum duration constraint which is instead typical of the OP. We refer to [3] and [15] for survey on deterministic routing problems with profits.

3. Related literature

The problem faced in this work lies in the realm of stochastic routing. Surveys on general stochastic routing problems are given by Bertsimas and Simchi-Levi [5], Gendreau et al. [19], and Cordeau et al. [12]. Here we review the literature most directly connected with our problem.

Tang and Miller-Hooks [29] consider a stochastic OP where service times at customers (or, equivalently, travel times) are stochastic. Additionally, each arc has a fixed, deterministic travel cost. The problem consists in finding a tour among a subset of nodes which maximizes the difference between collected prizes and travelling costs and such that the probability that the actual duration of the tour is above a certain limit does not exceed a given threshold. With respect to the POP, the objective is similar, but uncertainty is on the service or travel times, not on the availability of the customers. Consequently, the objective function in [29] is deterministic, and no recourse action is needed, though a hard chance constraint is added. In [29], an exact branch-and-cut method and a heuristic that constructs and adjusts solutions by a series of greedy procedures are proposed. Taylan et al. [22] consider an OP where the collected prizes are subject to uncertainty. The objective is to find a route covering a subset of nodes within a maximum time limit and maximizing the probability of collecting more than a specified target prize level. In their setting, the prize collected at a visited node is uncertain, but every node on the chosen route asks for service. As a consequence, no recourse strategy is needed and the total time is deterministic. Moreover, their objective is probabilistic, and involves only the collected prizes. Taylan et al. [22] model prizes as independent Normal random variables. This allows the reformulation of the problem as a deterministic bi-objective problem, where the objectives correspond to the average and the variance of the collected prize. For this problem, the authors propose a parametric exact algorithm and a genetic algorithm. Dye et al. [13] consider a two-stage stochastic OP where vehicle speed is uncertain and modelled by a finite set of constant-

speed scenarios. In the first stage, a single route is selected. In the second stage, the speed is revealed and the route may be shortened by heading directly back to base from any location on the route. If the time limit is exceeded, a penalty proportional to the delay is charged. The objective is to maximise the expected difference between the prizes collected at visited customers and the possible penalty. With respect to the POP, the uncertainty is not on the availability of the customers but on the speed of the vehicle, and the time limit can be exceeded. As a consequence, the recourse action and the objective are both different. In [13], a branch-and-bound algorithm with knapsack based bounds is developed. A stochastic OP with uncertain travel and service times is also studied by Campbell et al. [8]. In their setting, if a delivery commitment is completed on time then a prize is received, otherwise a penalty is incurred. Similarly to [13], the objective in [8] is the maximization of the expected difference between prizes and penalties. Differently from [13], no recourse action is allowed in [8], and uncertainty is modelled through general probability distributions. Polynomially solvable special cases are discussed and heuristics for the general problem are proposed. Another stochastic variant of the orienteering problem has been recently introduced by Evers et al. [14]. In this variant, the cost of each arc is uncertain, so that the actual cost of a tour is revealed only in the second stage of the problem. Nevertheless, the constraint about the total cost is hard: if the realized cost along the tour reaches the given bound, then the tour is aborted and the vehicle has to turn back to the depot. The second-stage recourse cost is then defined as the sum of the profits of the nodes in the first-stage tour that cannot be reached due to the specific cost realizations. The objective is to maximize the expected total prize. Similarly to the POP, the constraint about cost (or time) is hard. Differently from the POP, uncertainty is again on travelling cost, not on the availability of customers. As a consequence, the recourse action is different, and the objective includes expected prize evaluation. The authors propose a sample average approximation and a tailored heuristic, and perform computational tests to evaluate the performance of both approaches. Finally, Zhang et al. [38] consider a stochastic OP where a time window is associated with every customer, and the service time at each customer is related to the uncertain time spent in a queue. The structure of the problem is analyzed and a variable neighborhood search heuristic is developed. The value of the stochastic model is demonstrated comparing its solutions with corresponding solutions of a deterministic version of the model itself.

The probabilistic travelling salesman problem (PTSP) is a stochastic extension of the TSP where each node has to be visited with a certain probability, and the sequence of decisions and information is as follows. In the first stage, a Hamiltonian a priori tour covering all nodes has to be designed. In the second stage, information about the nodes to be visited is available, and the nodes are visited in the same order as they appear in the a priori tour by simply skipping the absent nodes. The problem consists in finding an a priori tour of minimum expected cost with respect to all possible subset of nodes to be visited. As in the POP, uncertainty is on the availability of the customers, and the recourse action is the same. Differently from the POP, the duration of the tour is not constrained and all customers must be included in the a priori tour, so that the expected total prize is constant and not included in the objective. The original formulation of the PTSP is due to Jaillet [23], who analyses the case where subsets of nodes with the same cardinality have the same probability. Under this hypothesis, a closed form expression for the computation of the expected cost of an a priori tour and several other results are given. Surveys on asymptotic results, approximations schemes, complexity theorems on a class of a priori combinatorial optimization problems including the PTSP are given by Bertsimas et al. [4] and Jaillet [24]. More recently, an empirical study of the distribution of optimal tour length for the PTSP is done by Vig and Palekar [33]. Laporte et al. [27] formulate the PTSP as a stochastic integer programming model and solve it through a branch-and-cut method which is a refinement of the integer L-shaped method proposed by Laporte and Louveaux [26] for general stochastic integer programs with complete recourse. They are able to solve at optimality instances with up to 50 nodes. More recently, Heilporn et al. [21] develop an L-shaped algorithm to find a minimum expected cost a priori tour for a single-vehicle Dial-a-Ride problem where customers may be absent with a certain probability.

Tang and Miller-Hooks [30] consider a probabilistic generalized TSP, which is an extension of the PTSP. As in the generalized TSP [16], the nodes are partitioned in a number of clusters, and in a first phase an a priori tour has to be designed which visits one node for every cluster. Every cluster is associated with a probability that at least one node in the cluster will actually require service. In the second phase, the tour is traversed skipping the clusters where no node requires service. The objective is to find an a priori tour with minimum expected travel time. For this problem, an integer L-shaped branch-and-cut method is proposed and three heuristics based on variations of constructive procedures for the TSP are suggested.

The probabilistic travelling salesman problem with deadlines (PTSPD) is a generalization of the PTSP where a specific deadline is associated with each customer. If a customer requiring service is reached after the deadline, a penalty is incurred. The objective is to find an a priori tour such that the expected cost plus the expected penalties for missed deadlines is minimized. The problem has been introduced by Campbell and Thomas [10], who present two recourse models and a chance constrained model and discuss several special cases. In PTSPD, the computation of the expected penalty associated with a given solution is very expensive. Heuristics for the approximate objective function evaluation are addressed in [9], [11], and [37], while a number of complexity results are given in [36] and [35].

Finally, Voccia et al. [34] present a recourse model and a variable neighborhood search algorithm for a PTSP with time windows.

To conclude, we remark that no problem is found in the literature with the same characteristics as the POP. Our approach is to consider the POP mainly as a generalization of the PTSP where: (1) origin and destination are different; (2) it is not mandatory to visit all customers and the total duration of the route is constrained; (3) the expected collected prize is included in the objective. We then build upon the literature on the PTSP and in particular on the work of Laporte et al. [27].

4. A MILP model

In this section, we formulate a stochastic MILP model for the POP and derive its deterministic equivalent, which will be used in the next section.

In order to describe a feasible path, we define two sets of binary variables. For all $i \in V$, variable y_i is 1 if and only if node i is included in the chosen path $\mathcal{P}$. Notice that $y_0 = y_{n+1} = 1$ in any feasible path. For all $(i, j) \in A$, variable x_{ij} equals 1 if and only if arc (i, j) is used by path $\mathcal{P}$. Given a node set $U \subseteq V$, let $A(U) \subseteq A$ denote the set of arcs with both endpoints in U, i.e., $A(U) = \{(i, j) \in A : i \in U, j \in U\}$.

A feasible path is then described by the following set of constraints:

$$y_0 = 1, \quad y_{n+1} = 1 \tag{5}$$

$$\sum_{j \in V \setminus \{0,i\}} x_{ij} = y_i \quad (i \in V \setminus \{n+1\}) \tag{6}$$

$$\sum_{j\in V\setminus\{0,i\}} x_{ij} = \sum_{j\in V\setminus\{i,n+1\}} x_{ji} \quad (i \in V\setminus\{0, n+1\}) \tag{7}$$

$$\sum_{(i,j)\in A(U)} x_{ij} \le \sum_{h\in U\setminus\{k\}} y_h \quad (U \subseteq V, k \in U) \tag{8}$$

$$\sum_{(i,j)\in A} t_{ij}x_{ij} \le D_{\max} \tag{9}$$

$$y_i \in \{0,1\} \quad (i \in V) \tag{10}$$

$$x_{ij} \in \{0,1\} \quad ((i,j) \in A) \tag{11}$$

Constraints (5) imply that the server must visit both 0 and $n+1$; constraints (6) impose that $y_i = 1$ if and only if the chosen path visits customer i; constraints (7) are flow conservation constraints; constraints (8) are subtour elimination constraints; constraint (9) imposes the maximum duration of the path.

Let XY denote the set of binary vectors $(\mathbf{x}, \mathbf{y})$ with $\mathbf{x} = (x_{ij} : (i,j) \in A)$ and $\mathbf{y} = (y_i : i \in V)$ satisfying constraints (5)–(11). Due to constraints (6), variables y could be eliminated, but we decided to maintain them for the ease of notation.

The objective function, to be maximized, is the difference between the expected total prize and the expected total cost. A linear expression for the expected total prize is easily obtained from (1) and (6). We have

$$\mathbb{E}[P(\mathcal{P})] = \sum_{i\in V} \pi_i\, p_i\, y_i = \sum_{i\in V\setminus\{n+1\}} \pi_i\, p_i \sum_{j\in V\setminus\{0,i\}} x_{ij} = \sum_{(i,j)\in A} \pi_i\, p_i\, x_{ij}. \tag{12}$$

Unfortunately, Eq. (4) cannot be used to obtain a linear expression for the expected total cost. We then extend the approach used in [27] to express the expected cost of a tour in the PTSP.

Once a vector $(\mathbf{x}, \mathbf{y}) \in XY$ is given, the maximum duration induced by $\mathbf{x}$ is expressed as

$$T_{\max}(\mathcal{P}) = \sum_{(i,j)\in A} t_{ij}\, x_{ij}. \tag{13}$$

The a priori solution $(\mathbf{x}, \mathbf{y})$ may be modified in the second stage, according to the realization of the random vector $\mathbf{b}$: all customers i such that $b_i = 0$ are skipped while maintaining the order of visit for the remaining customers. Let $R(\mathbf{x}, \mathbf{y}, \mathbf{b})$ be the time reduction on $T_{\max}(\mathcal{P})$ obtained by the skipping process when scenario $\mathbf{b}$ is realized, and let $R(\mathbf{x}, \mathbf{y})$ be the expected value with respect to all possible scenarios, i.e., $R(\mathbf{x}, \mathbf{y}) = \mathbb{E}[R(\mathbf{x}, \mathbf{y}, \mathbf{b})]$. We may write $\mathbb{E}[T(\mathcal{P})] = T_{\max}(\mathcal{P}) - R(\mathbf{x}, \mathbf{y})$.

To be consistent with the approach in [27], we define $Q(\mathbf{x}, \mathbf{y}) = -R(\mathbf{x}, \mathbf{y})$ (i.e., $Q(\mathbf{x}, \mathbf{y})$ is the "expected recourse time"), and we cast our model in minimization form. Thus our objective is

$$\min\ C\mathbb{E}[T(\mathcal{P})] - \mathbb{E}[P(\mathcal{P})] = \sum_{(i,j)\in A} (Ct_{ij} - \pi_i p_i)x_{ij} + CQ(\mathbf{x}, \mathbf{y}). \tag{14}$$

A stochastic integer linear programming formulation of the POP is thus obtained minimizing the above function subject to constraints (5)–(11). We can however obtain a deterministic equivalent MILP model.

Let

$$(\mathbf{x}^1, \mathbf{y}^1), (\mathbf{x}^2, \mathbf{y}^2), \ldots, (\mathbf{x}^K, \mathbf{y}^K)$$

be the binary vectors identifying all possible feasible paths $\mathcal{P}$, with $K = |XY|$, i.e., let $XY = \{(\mathbf{x}^k, \mathbf{y}^k) : k = 1, \ldots, K\}$, and notice that $Q(\mathbf{x}^k, \mathbf{y}^k) \ge -D_{\max}$ for all $k = 1, 2, \ldots, K$. Let $A^k = \{(i,j) \in A : x^k_{ij} = 1\}$ for all $k = 1, 2, \ldots, K$, and define the following function

$$f^k(\mathbf{x}) = Q(\mathbf{x}^k, \mathbf{y}^k) - \left(Q(\mathbf{x}^k, \mathbf{y}^k) + D_{\max}\right)\left(1 + \sum_{i\in V} y^k_i - \sum_{(i,j)\in A^k} x_{ij}\right). \tag{15}$$

Notice that in the above function $\mathbf{x}^k$ and $\mathbf{y}^k$ are constant vectors, whereas $\mathbf{x}$ is the variable vector. It is easy to see that

$$f^k(\mathbf{x}) \begin{cases} = Q(\mathbf{x}^k, \mathbf{y}^k) & \text{if } \mathbf{x} = \mathbf{x}^k, \\ \le -D_{\max} & \text{if } \mathbf{x} \ne \mathbf{x}^k. \end{cases}$$

We then have the following deterministic equivalent MILP model for the POP, where the scalar variable θ is used to describe the value of $Q(\mathbf{x}, \mathbf{y})$.

$$\min \sum_{(i,j)\in A} (Ct_{ij} - \pi_i p_i)x_{ij} + C\theta \tag{16}$$

$$\text{subject to } \theta \ge f^k(\mathbf{x}) \quad (k = 1, \ldots, K) \tag{17}$$
$$\text{constraints (5)–(11)}$$

In the following, we call $\mathcal{F}$ the formulation given by (16), (17), and (5)–(11). Formulation $\mathcal{F}$ has $O(n^2)$ binary variables but an exponential number of constraints. In particular, both constraint sets (8) and (17) have exponential cardinality. Hence, the model requires a branch-and-cut method to be solved. In particular, constraints (17) extend the *optimality cuts* used by Laporte et al. [27] for the probabilistic TSP and introduced by Laporte and Louveaux [26] for general integer programs. A separation algorithm for these constraints corresponds to a method to compute $Q(\mathbf{x}, \mathbf{y})$ or, equivalently, the expected cost of a given feasible path. These issues will be discussed in detail in the next section.

5. A branch-and-cut algorithm

We propose a branch-and-cut algorithm to find an optimal solution in $\mathcal{F}$. Constraints (8) and (17) are initially relaxed and introduced only when violated.

We now describe in details the different components of the algorithm. In particular, Sections 5.1 and 5.2 are devoted to the description of the separation algorithms of the subtour elimination constraints and the optimality cuts, respectively. In Section 5.3, lower bounds on the value of θ are discussed. Section 5.4 describes the ad-hoc branching rules we propose, which constitute the main innovative contribution of the branch-and-cut algorithm. As shown in the computational results section, they are proved to be much more effective then the default branching rules applied by the commercial solver we used for testing.

5.1. Separation of the subtour elimination constraints

For the separation of the subtour elimination constraints (8), we use the classical separation algorithm introduced in [28] which consists in solving a max-flow problem on the graph induced by the value of the $\mathbf{x}$ variables. If a subtour is detected covering a set $U \subseteq V$, we introduce the violated inequality (8) with $k = \text{argmax}\{y_h : h \in U\}$. In addition, when the solution is integer, one of the subtours consists in a path going from the origin to the destination which is a feasible solution for the POP. Thus, this path is used to possibly update the current best feasible solution found. Finally, we mention that subtour elimination constraints on fractional solutions are separated until a certain level of the branch-and-bound tree. This choice is aimed at reducing the computational burden induced by separating the subtour elimination constraints at all nodes. In fact, subtour elimination constraints tend to be effective at the root node and at the first levels of the branch-and-bound tree, while the benefit of their introduction, in terms of improvement of the lower bound, decreases when the search reaches deeper levels.

5.2. Separation of the optimality cuts

The procedure to separate optimality cuts is inherited from the one described in [27]. We thus provide a short description and refer the reader to [27] for more details.

Optimality cuts (17) are separated only when the feasible solution $(\mathbf{x}^\star, \mathbf{y}^\star)$ defines a path $\mathcal{P}^\star : i_0, i_1, i_2, \ldots, i_q, i_{q+1}$ from $i_0 = 0$ to $i_{q+1} = n+1$ visiting q customers, with $i_h \in V'$ for $h = 1, 2, \ldots, q$. In this case, we derive the value

$$Q(\mathbf{x}^\star, \mathbf{y}^\star) = \mathbb{E}[T(\mathcal{P}^\star)] - T_{\max}(\mathcal{P}^\star),$$

where $\mathbb{E}[T(\mathcal{P})]$, computed as in (13), is the expected time of the path, and $T_{\max}(\mathcal{P})$, computed as in (2), is the a priori cost of the path.

5.3. Lower bounds on the value of θ

In order to speed-up the solution process, we provide different lower bounds on the value of θ. Two trivial lower bounds are provided by the following inequalities:

$$\theta \geq -D_{\max}, \quad \theta \geq -\sum_{(i,j)\in A} t_{ij} x_{ij}. \tag{18}$$

A further bound is the one proposed in [27]. We provide a full description of the procedure used to compute it as it has been adapted to deal with the directed case, while in [27] the authors consider an undirected graph. The bound is the following:

$$\theta \geq \sum_{(i,j)\in A} (\delta_{ij} - t_{ij}) x_{ij} \tag{19}$$

where $\delta_{ij} = \pi_i \pi_j t_{ij} + \pi_i (1-\pi_j) \min_{k \notin \{i,j\}} t_{ik}$. Bound (19) is valid because $\sum_{(i,j)\in A} \delta_{ij} x_{ij}$ is a lower bound on the expected time required by $(\mathbf{x}, \mathbf{y})$. We refer to [27] for more details.

It is also possible to reinforce the previous lower bounds by considering chains composed of arcs whose corresponding x variable has been fixed to 1 by the branching constraints. In particular, let $\mathcal{R} : i_1, i_2, i_3, \ldots, i_q$ be a chain, i.e., a maximal sequence of vertices for which the arcs (i_k, i_{k+1}) are associated with an x variable fixed to 1. Note that we assume neither that $i_1 = 0$ nor that $i_q = n+1$. When $q = 1$, we have a degenerate chain formed by a single vertex and no arcs. We can compute a lower bound H on the expected travelling time of the arcs traversed in the chain as follows:

$$H = \sum_{h=1}^{q-1} \pi_{i_h} \left[\sum_{k=h+1}^{q} \left(t_{i_h i_k} \pi_{i_k} \cdot \Pi_{l=h+1}^{k-1} (1 - \pi_{i_l}) \right) \right].$$

Moreover, let $\overline{A}$ be the subset of arcs which can be included in the a priori path given the arcs traversed in chain $\mathcal{R}$. The subset $\overline{A}$ is obtained from A by removing:

- all arcs incident to vertices whose corresponding y variable is fixed to 0;
- all arcs exiting from vertices i_h, $h = 1, \ldots, q-1$;
- all arcs entering in vertices i_h, $h = 2, \ldots, q$;
- the arcs joining a chain starting in 0 with a chain ending in $n+1$ when there is at least another chain.

All the above arcs are removed from A as their traversal induces an infeasible solution.

The lower bound on the value of θ is thus given by:

$$\theta \geq H + \sum_{(i,j)\in \overline{A}} (\delta_{ij} - t_{ij}) x_{ij}.$$

When more than one chain is present in the current solution, the value of H is given by the sum of the H values corresponding to each chain, while the subset $\overline{A}$ is the intersection of all subsets $\overline{A}$ related to each chain.

The value of H can be easily updated when a new arc is added to a chain. In particular, if an arc (i_q, i_{q+1}) is added at the end of a chain, then the new value of H is:

$$\overline{H} = H + \pi_{i_{q+1}} \sum_{h=0}^{q} \left[t_{i_h i_{q+1}} \pi_{i_h} \cdot \Pi_{l=h+1}^{q} (1 - \pi_{i_l}) \right].$$

If instead an arc (i_0, i_1) is added at the beginning of a chain, then the new value of H is:

$$\overline{H} = H + \pi_{i_0} \sum_{h=1}^{q} \left[\left(t_{i_0 i_h} \pi_{i_h} \cdot \Pi_{l=1}^{h-1} (1 - \pi_{i_l}) \right) \right].$$

Finally, if an arc (i_q, j_1) is added which joins two chains $\mathcal{R}' : i_1, i_2, i_3, \ldots, i_q$ and $\mathcal{R}'' : j_1, j_2, j_3, \ldots, j_s$, then the new value of H is calculated as follows. Let H' and H'' be the values of H associated with $\mathcal{R}'$ and $\mathcal{R}''$. The value of H associated with the chain $\mathcal{R} : i_1, \ldots, i_q, j_1, \ldots, j_s$ is given by:

$$\overline{H} = H' + H'' + \pi_{i_q} \pi_{j_1} t_{i_q, j_1} + \pi_{i_q} \sum_{h=2}^{s} \left[\left(t_{i_q j_h} \pi_{j_h} \cdot \Pi_{l=1}^{h-1} (1 - \pi_{j_l}) \right) \right] + \pi_{j_1} \sum_{h=1}^{q-1} \left[\left(t_{i_h j_1} \pi_{i_h} \cdot \Pi_{l=h+1}^{q} (1 - \pi_{i_l}) \right) \right].$$

5.4. Branching rules

We propose ad-hoc branching rules for the POP. This fact differentiates our approach from other L-shaped methods proposed in the literature and is the most innovative contribution in our exact method. More specifically, the branch-and-cut method proposed in [27] for the PTSP is focused on bounding procedures and uses generic branching rules. Similarly, the method proposed in [21] for a stochastic Dial-a-Ride problem does not use specific branching rules.

Essentially, POP consists in finding the best path from vertex 0 to vertex $n+1$. Thus, we propose three different rules which are aimed at focusing on chains starting in 0 or ending in $n+1$. The rules not only establish the variable (or variables) that must be fixed to either 0 or 1, but also update the value of θ according to the rules defined in Section 5.3. Notice that the chain concept has been introduced in [27], but used there only in connection with lower bounding.

5.4.1. Chain

The variable on which the branching is performed is chosen by the MILP solver with the default rule. Nothing is made except for the case when an x variable is fixed to 1. In this case, a new chain is created, or a previous chain is prolonged, or two previous chains are joined. As a consequence, we update the subset $\overline{A}$ and we set to 0 the values of the x variables of all arcs in $A \backslash \overline{A}$.

5.4.2. Path

The branching is performed only on x variables. More precisely, the branching is performed on arcs prolonging the path exiting from the origin 0 and choosing the variable with the highest value. In the branch where the variable is fixed to one, as done in the **Chain** rule, we update the subset $\overline{A}$ and we set to 0 the values of the x variables of all arcs in $A \backslash \overline{A}$. Moreover, we set to 0 all y variables associated with customers that cannot be feasibly reached, as they imply a violation of the $D_{\max}$ constraint. Nothing is made when the x variable is fixed to 0.

5.4.3. Two-ways-path

This rule is similar to the previous one with the exception that two paths are prolonged: the one exiting from the origin 0, which is prolonged forward, and the one entering in the destination $n+1$, which is prolonged backward. The variable with the highest value prolonging one of the two paths is chosen for branching. The same procedure as in the **Path** rule is applied when the variable is fixed to one.

We mention that we also made some tests with another branching rule not listed above which is a generalization of the **Path** rule where a couple of arcs, instead of a single arc, is considered in each branch, i.e., we look at the sum of the variables related to two arcs prolonging the path exiting from the depot. However, this rule did not produce promising results so we discarded it.

6. Matheuristic algorithm

Matheuristics are becoming more and more popular solution tools to tackle combinatorial optimization problems. Recently, they have been widely applied for the solution of routing problems (see [2] for details). Following the definition reported in [7], 'matheuristics are heuristic algorithms made by the interoperation of metaheuristics and mathematical programming techniques'.

Given the success that matheuristics have achieved in the class of routing problems, we decided to use them to get good solutions for the POP in a reasonable amount of computing time. In particular, our matheuristic scheme is based on the exact branch-and-cut algorithm presented in the previous section. Our idea is to exploit as much as possible the potentiality of the exact algorithm by applying it to a subset of the solution space which is more promising. Thus, we heuristically discard subsets of solutions which do not seem to contain high quality solutions, and we apply the branch-and-cut algorithm on the remaining (and more promising) subset of solutions.

The basic scheme of our matheuristic algorithm is the same as the one of the exact algorithm described in the previous section, i.e., the matheuristic is a branch-and-cut algorithm where we apply heuristic rules to reduce the solution space. In particular, the heuristic rules we applied are the following:

- *Heuristic branching*: we heuristically remove branches of the branch-and-bound tree that lead to unpromising solution subsets.
- *Variable fixing*: we heuristically fix the value of subsets of variables. Again, we focus on those variables related to unpromising solution subsets.

The main characteristic of our matheuristic scheme is that it is able to provide feasible solutions in a much shorter computing time with respect to the exact branch-and-cut algorithm. Moreover, it is easy to design and implement it once the exact algorithm has been built. In fact, our main purpose is to take advantage of the exact algorithm by applying easy modifications which enable to obtain heuristic solutions quickly. As mentioned in [2], the idea of modifying exact branch-and-cut (or branch-and-price) algorithms in order to find heuristic solutions has been widely and successfully applied, especially for complex problems for which the design of ad-hoc heuristic schemes is difficult, as is the case for the POP.

We now describe in detail the two heuristic rules.

6.1. Heuristic branching

Heuristically discarding some branches of the branch-and-bound tree is one of the most natural ways to reduce the solution space. The key point is to choose to discard those branches which are likely not to produce high quality solutions. We apply the idea as follows. A counter η_i is assigned to each vertex $i \in V$ and is initialized at 0. Each time a variable x_{ij} or x_{ji} are fixed at 0 in a branch of the branch-and-bound tree, the counter η_i is augmented by 1. When $\eta_i \geq \Omega$, all branches for which x_{ij} or x_{ji} are fixed to 0 are immediately fathomed. The rationale is that the Ω most promising arcs incident to vertex i has already been set, either to 0 or 1, in previous branching operations. Thus, setting a further arc incident to i at 0 will leave the choice of setting to 1 one of the remaining $n+2-(\Omega+1)$ arcs, which are less promising.

6.2. Variable fixing

The second heuristic rule we apply is variable fixing, i.e., we heuristically fix the value of subsets of variables. We apply this procedure both to x and y variables. The idea is that, if a variable takes value 0 in the optimal solution of the relaxed problem at a node of the branch-and-bound tree for a given number of times α, then this value can be reasonably fixed to 0 as it is likely to be such in high quality solutions. Moreover, in the case of the x variables, the choice of the subset of variables to be set to 0 is also based on a ranking of the arcs. In particular, we rank all arcs on the basis of a non-increasing value of the travelling time t_{ij} and we partition them in β classes with the same cardinality. Let $\beta_{ij} \in \{1, \ldots, \beta\}$ be the class of arc $(i, j) \in A$. Then, variable x_{ij} is fixed to 0 if it has taken value 0 in the optimal solution of a node of the branch-and-bound tree for a number of times equal to the minimum between α and β_{ij}. The idea is that long arcs belonging to the first class are more unlikely to be part of high quality solutions than short arcs and thus can be discarded earlier.

The heuristic variable fixing is applied up to a given level γ of the branch-and-bound tree. Once level γ is reached, it is no longer applied, thus the value of the variables is fixed on the basis of the branching rules only. The motivation of this choice is that variable fixing has a high impact at the very first levels of the branch-and-bound tree. The efficacy of the procedure, in terms of reduction of the solution space, decreases as the depth of the tree increases. Thus, we decided to abandon its application once the level γ is reached in favor of the possibility of obtaining better solutions.

7. Computational tests

In the following, we present the computational tests we made in order to evaluate the performance of the algorithms presented in Sections 5 and 6. All tests were performed on a 64-bit Windows machine, with Intel Xeon processor E5-1650, 3.50 GHz, and 16GB of RAM. CPLEX 12.6 (64 bit version) was used as exact MILP solver. The code was written in Java 8. The maximum running time has been set to 7200 s both for the branch-and-cut and the matheuristic algorithms. A single thread was used in all experiments. The test instances and the results for both the branch-and-cut and the matheuristic algorithms are available at http://or-brescia.unibs.it/instances.

7.1. Test instances

To the best of our knowledge, the POP has never been studied before in the literature and no benchmark instances are available. Thus, to test the performance of the branch-and-cut and the matheuristic algorithms presented in Sections 5 and 6, we generated a set of benchmark instances as follows. We take the TSP benchmark instances from the TSPLIB95 library available at the following url: http://comopt.ifi.uni-heidelberg.de/software/TSPLIB95. We consider all instances with a number of vertices ranging from 14 to 99, for a total of 22 instances. We keep the data concerning

the location of the vertices. We number the vertices of the original instance from 0 to n, and we create a dummy vertex $n+1$ as a copy of 0. We then consider vertex 0 as the origin, vertices from 1 to n as customers, and vertex $n+1$ as destination. The remaining problem characteristics are set as follows:

- The value of $D_{\max}$ is set as $\omega \cdot TSP^*$ where TSP^* is the optimal value of the TSP over all vertices, obtained by solving the corresponding TSP instance with the Concorde solver available at www.math.uwaterloo.ca/tsp/concorde.html. We considered three values of ω: 1/4, 1/2, and 3/4.
- The prizes are generated according to two rules as done in [17] for the OP. The first rule sets the prize of each vertex equal to 1. The second rule sets the prize of a vertex i as an integer pseudo-randomly generated in $\{1, 2, \ldots, 100\}$ by the formula $1 + (7141i + 73) \bmod 100$.
- After some preliminary tests and in order to obtain a reasonable trade-off between expected prizes and expected costs in the objective function, the cost per unit time C is set equal to 10^{-3}.
- Probabilities π_i are set according to two rules. In the first rule, $\pi_i = 0.5$ for each $i \in V$. In the second rule, π_i is equal to a random number in the interval [0.25, 0.75], $i \in V$.

For each of the 22 TSP benchmark instances considered, we generated a different POP instance for each value of ω, prize generation scheme and interval for probabilities π_i. As we considered 3 values for ω, 2 prize generation schemes and 2 intervals for π_i, we generated a total number of 264 instances.

7.2. Computational results

We now present the results of the computational tests on the instances presented in Section 7.1. We first focus on the performance of the branch-and-cut algorithm and then show the results of the matheuristic algorithms.

7.2.1. Branch-and-cut algorithms

We tested four variants of the branch-and-cut algorithm presented in Section 5. Each variant contains all procedures described in Sections 5.1–5.3 and differs in terms of the branching rule adopted. In particular, we tested the variants related to the three branching rules described in Section 5.4 and the one where the CPLEX default branching rule is used. In the following, we call each variant with the name of the corresponding branching rule adopted, i.e, `Chain`, `Path`, `Two-ways-path`, and `Basic` for the version with the CPLEX default branching rule. The subtour elimination constraints on fractional solutions are separated till the sixth level of the branch-and-bound tree, except for the `Basic` variant where they are separated only on integer solutions.

The results are summarized in Tables 2–4. Table 2 reports the number of instances solved to optimality within the time limit by each algorithm. We partition the 264 instances on the basis of four different criteria: the number n of customers; the value of ω; the kind of generation of profits; the value of the probability π_i. The last row of the table reports the results over all instances. The second column reports the number of instances in the class of the corresponding row. Table 3 is structured as Table 2 and refers to the average percentage optimality gap at the end of the computation, while Table 4 reports the average computing time in seconds. Note that each row of Tables 2–4 reports the results related to all instances with the characteristic identified in the first column. For example, the row $\omega = 1/4$ refers to the 88 instances generated from the 22 TSP benchmark instances, with both prize generation schemes and considering the two intervals for the generation of π_i as described in Section 7.1.

Table 2
Number of instances solved to optimality.

	# instances	`Basic`	`Chain`	`Path`	`Two-ways-path`
$n < 28$	84	63	67	67	67
$28 \le n < 48$	84	58	66	68	69
$48 \le n \le 98$	96	25	62	61	62
$\omega = 1/4$	88	62	78	81	83
$\omega = 1/2$	88	46	66	66	66
$\omega = 3/4$	88	38	51	49	49
$p_i = 1$	132	63	77	78	80
$p_i \in \{1, \ldots, 100\}$	132	83	118	118	118
$\pi_i = 0.5$	132	71	95	97	98
$\pi_i \in [0.25; 0.75]$	132	75	100	99	100
Total	**264**	**146**	**195**	**196**	**198**

Table 3
Average percentage optimality gap.

	# instances	`Basic`	`Chain`	`Path`	`Two-ways path`
$n < 28$	84	4.06	2.55	2.47	2.37
$28 \le n < 48$	84	2.79	1.79	1.52	1.48
$48 \le n \le 98$	96	23.89	7.27	7.39	7.35
$\omega = 1/4$	88	11.62	2.77	2.44	2.27
$\omega = 1/2$	88	11.32	4.01	3.96	3.98
$\omega = 3/4$	88	9.45	4.82	4.89	4.85
$p_i = 1$	132	13.16	7.65	7.40	7.27
$p_i \in \{1, \ldots, 100\}$	132	8.43	0.08	0.13	0.13
$\pi_i = 0.5$	132	9.91	3.94	3.80	3.73
$\pi_i \in [0.25; 0.75]$	132	11.68	3.80	3.72	3.67
Total	**264**	**10.79**	**3.87**	**3.76**	**3.70**

Table 4
Average computational time (seconds).

	# instances	`Basic`	`Chain`	`Path`	`Two-ways-path`
$n < 28$	84	2024	1606	1516	1500
$28 \le n < 48$	84	2540	1443	1274	1142
$48 \le n \le 98$	96	5678	2978	3213	3144
$\omega = 1/4$	88	2479	899	806	572
$\omega = 1/2$	88	3775	1958	2010	1981
$\omega = 3/4$	88	4504	3366	3366	3405
$p_i = 1$	132	4162	3309	3136	2989
$p_i \in \{1, \ldots, 100\}$	132	3010	840	985	983
$\pi_i = 0.5$	132	3674	2115	2123	2025
$\pi_i \in [0.25; 0.75]$	132	3498	2034	1999	1947
Total	**264**	**3586**	**2074**	**2061**	**1986**

The results show that the branching rules we proposed are clearly effective as they improve the performance of the algorithm. In fact, the `Chain`, `Path`, and `Two-ways-path` outperforms the `Basic` variant on all measures. Among them, the `Two-ways-path` version is the best one with a higher number of instances solved to optimality, a lower optimality gap and a lower computational time. The dominance of the `Chain`, `Path`, and `Two-ways-path` variants with respect to the `Basic` variant is due to the fact that the former three variants exploit the structure of the problem, while this is not the case in the last variant. For the same reason, `Path` and `Two-ways-path` variants perform slightly better than the `Chain` variant. Among the former three variants, `Two-ways-path` performs better as it extends the path in both directions. Thus, at each branching iteration, it considers a wider range of choice with respect to `Path` variants, and this leads to a better performance. We note that, when $\omega = 3/4$, `Path` and `Two-ways-path` variants perform slightly worse than the `Chain` variant, both in terms of number of instances solved to optimality and in terms of average optimality gap. The difference is not relevant. Anyway, the performance drop of the `Path` and

Two-ways-path for these classes of instances may be due to the wider solution space related to a larger value of $D_{\max}$ which may counter-balance the benefits of choosing properly the variables on which the branching is performed.

Concerning the number of instances solved to optimality (Table 2), the Basic variant solves 55% of the instances while the percentage goes up to 74% and 75% for the other variants. The difficulty of the problem increases with the number of customers, as expected, but the trend is not monotonic for all variants. What is interesting to note is that the three variants Chain, Path, and Two-ways-path are much less sensitive to the number of customers than the Basic variant. In fact, the percentage of instances solved to optimality goes from 75% when $n < 28$ to 26% when $n \geq 48$ for the Basic variant, while for the other three variants it goes from 80% when $n < 28$ to 64% when $n \geq 48$. This may be due to the fact that an effective branching rule, as the one implemented in the Path and Two-ways-path variants, helps in pruning branch-and-bound nodes earlier and thus mitigate the effect of the increasing complexity related to the larger number of customers. Regarding the Chain variant, the same effect is due to the chains updating. Concerning the value of $D_{\max}$ and the kind of generation of profits, there is a clear evidence that the problem difficulty increases when the value of $D_{\max}$ increases and when the customer profits are all set to 1. In particular, focusing on the three best variants, the number of instances solved goes from 91%–97% when $\omega = 1/4$ to 57%–59% when $\omega = 3/4$, on one side, and from 58%–61% when $p_i = 1$ to 89% when $p_i \in \{1, \ldots, 100\}$. This behavior may be explained as follows. An increasing value of $D_{\max}$ is related to a wider solution space which makes the problem more difficult to solve. Concerning profits generation, when they are all set to 1 then we obtain a more flat range of objective values of feasible solutions and this makes it more difficult to cut off unpromising areas of the solution space. In fact, comparing the subset of instances with $p_i = 1$ and $\pi_i = 0.5$ for all customers with the subset of instances with variable values of p_i and π_i we have that the number of instances solved to optimality by the four variants of the algorithm is 30, 36, 38 and 40, respectively, for the first subset, while it is 42, 59, 59 and 60 for the second subset. Interestingly enough, this difference seems not to depend on the way the values of π_i are generated. In particular, the number of instances solved for the case where π_i varies in the interval [0.25, 0.75] is from 2% to 5% higher than in the case where all π_i are set equal to 0.5.

Concerning the average percentage optimality gap (Table 3) and the computational time (Table 4), the considerations made on Table 2 are still valid. In particular, we note, from Table 4, that the Chain variant is slightly slower than the Path and Two-ways-path variants. This may be related to a less effective branching rule.

To give statistical support to the above claims, we apply the non-parametric Friedman test [18]. We first compare the number of instances solved to optimality by Basic, Chain, Path, and Two-ways-path in Table 2. The null hypothesis H_0 is that none of the techniques is better than another. The alternative hypothesis H_1 is that there is a difference among the four procedures. We obtain a chi-square value of 23.329 with 3 degrees of freedom and a p-value of 3.448×10^{-5}. This means that H_0 can be rejected. If we restrict the test to Chain, Path, and Two-ways-path, we obtain a chi-square value of 4.88 with 2 degrees of freedom and a p-value of 0.08716. This means that if we reject H_0 (no difference among our branching rules) then the probability of error is around 8.7%. Performing the same analysis on the figures in Tables 3 and 4, we get analogous results.

7.2.2. Matheuristic algorithms

We tested three variants of the matheuristic scheme presented in Section 6, one for each of the branching rules presented in Section 5.4. We call them Smart-Chain, Smart-Path, and Smart-Two-ways-path, respectively.

We performed a number of preliminary tests in order to find the best value of Ω, α, β and γ. We recall that Ω is the parameter used in the heuristic branching rule (Section 6.1) which identifies which branches are heuristically fathomed. Parameters α, β and γ are instead used for the heuristic variable fixing (Section 6.2): α is the parameter used to identify which variables are set to 0, β determines the number of classes in which arcs are classified, and γ is the level of the branch-and-bound tree up to which the heuristic variable fixing is applied. On the basis of the results we obtained, we set the value of Ω and β to 4. Concerning the value of α and γ, the best results were obtained by fixing $\alpha = 2$ and $\gamma = 4$ when considering the y variables and $\alpha = 4$ and $\gamma = 8$ for the x variables. We mention that the subtour elimination constraints on fractional solutions are separated till the eighth level, contrary to the case of the branch-and-cut algorithms where they are separated until the sixth level. The reason is that we need to guarantee that the graph remains connected when fixing the x variables to 0, and this is guaranteed through the separation of the subtour elimination constraints.

Results of the final tests are shown in Tables 5–8, which have a structure similar to Tables 2–4. In Table 5, we report the average and maximum percentage errors, calculated as follows: for each instance, we take the value of the solution provided by the algorithm we are considering (z) and the value of the best solution found for the same instances by all algorithms we tested, both exact and heuristic (z_{best}). In particular, to determine the value of z_{best}, we consider all variants tested also during preliminary tests which we do not describe for the sake of brevity. The percentage error is given by $100 \cdot \frac{z - z_{best}}{z_{best}}$. Table 6 gives the number of times the corresponding algorithm has found the best solution. Table 7 refers to the average computational time while Table 8 reports the average time to incumbent, i.e., the average time at which the final (and thus best) solution has been found.

Looking at Table 5, we see that the algorithms perform well as the average error is below 1% for all. The maximum error goes up to 12.22% for Smart-Chain, 20.01% for Smart-Path and 15.34% for Smart-Two-ways-path. However, note that if we exclude the class of instances with $\omega = 1/4$ then the maximum error is below 10% for all algorithms. This is due to the fact that, when $\omega = 1/4$, the solution space is very narrow and either the matheuristic finds the best solution, or the second best can be far from the best solution. Considering the number of customers, the trend of the average and maximum error is not monotonic. No significant remarks are related to the kind of generation of profits and of the probabilities π_i. In general, Smart-Chain is the best version as it always gives the smallest errors. This is confirmed by the Friedman test applied to the average errors of Smart-Chain, Smart-Path, and Smart-Path-two-ways with null hypothesis H_0 that no method has different errors than another. We obtain a chi-squared value of 15.8 with 2 degrees of freedom and a p-value of 0.0003707. The superiority of Smart-Chain is confirmed also from Table 6 where we can see that Smart-Chain is the algorithm that provides the highest number of best solutions. Focusing on the Smart-Chain version, we note that the number of best solutions decreases when the number of customers increases and is much higher in class $p_i \in \{1, \ldots, 100\}$ than in class $p_i = 1$. Also, when the value of $D_{\max}$ is less binding, it is more difficult for all heuristics to find the best solution. This is due to the fact that the solution space is wider and thus the number of feasible solutions is higher. Thus, the heuristic algorithms find good quality solutions but have more difficulties in finding the best one. The Friedman test applied to Table 6, with null hypothesis H_0 that no method finds more best solutions than another, produces a chi-

Table 5
Average and maximum percentage errors.

		Average			Maximum		
	# instances	Smart Chain	Smart Path	Smart-Path two-ways	Smart Chain	Smart Path	Smart-Path two-ways
$n < 28$	84	2.52	3.93	2.72	12.22	7.67	8.21
$28 \leq n < 48$	84	0.38	0.75	1.05	6.00	20.01	15.34
$48 \leq n \leq 98$	96	0.77	1.17	1.34	8.81	9.09	8.80
$\omega = 1/4$	88	1.024	1.254	1.567	12.22	20.01	15.34
$\omega = 1/2$	88	0.228	0.632	0.579	3.73	7.67	8.21
$\omega = 3/4$	88	0.353	0.610	0.653	4.96	8.74	6.88
$p_i = 1$	132	0.78	0.88	1.11	12.22	20.01	13.57
$p_i \in \{1, \ldots, 100\}$	132	0.29	0.78	0.75	5.71	9.09	15.34
$\pi_i = 0.5$	132	0.37	0.72	0.87	8.489	9.093	15.344
$\pi_i \in [0.25; 0.75]$	132	0.70	0.94	1.00	12.220	20.012	11.358
Total	**264**	**0.53**	**0.83**	**0.93**	**12.22**	**20.01**	**15.34**

Table 6
Number of best solutions found.

	# instances	Smart-Chain	Smart-Path	Smart-Two-ways-path
$n < 28$	84	63	56	60
$28 \leq n < 48$	84	49	37	34
$48 \leq n \leq 98$	96	41	26	23
$\omega = 1/4$	88	56	50	48
$\omega = 1/2$	88	54	37	39
$\omega = 3/4$	88	43	32	30
$p_i = 1$	132	65	58	54
$p_i \in \{1, \ldots, 100\}$	132	88	61	63
$\pi_i = 0.5$	132	82	58	61
$\pi_i \in [0.25; 0.75]$	132	71	61	56
Total	**264**	**153**	**119**	**117**

Table 7
Average computational time (seconds).

	# instances	Smart-Chain	Smart-Path	Smart-Two-ways-path
$n < 28$	84	286	282	345
$28 \leq n < 48$	84	482	362	309
$48 \leq n \leq 98$	96	1432	1264	1375
$\omega = 1/4$	88	278	219	209
$\omega = 1/2$	88	544	535	514
$\omega = 3/4$	88	1472	1241	1380
$p_i = 1$	132	1300	1180	1188
$p_i \in \{1, \ldots, 100\}$	132	229	150	214
$\pi_i = 0.5$	132	793	726	717
$\pi_i \in [0.25; 0.75]$	132	737	604	685
Total	**264**	**765**	**665**	**701**

squared value of 15.2 with 2 degrees of freedom and a p-value of 0.0005005. This suggests to reject the null hypothesis.

Concerning the computational time, we note that Smart-Chain is the more time consuming version, even if the difference with the other two versions is not large. In fact, the Friedman test applied to Table 7, with null hypothesis H_0 that no method is more time consuming than another, produces a chi-squared value of 12.6 with 2 degrees of freedom and a p-value of 0.001836. This suggests to reject the null hypothesis with an error probability smaller than 0.2%. The computational time increases with the number of customers. The relation is monotonic for all versions of the algorithm also when considering the value of ω. Moreover, class $p_i = 1$ requires a much higher computing time than class $p_i \in \{1, \ldots, 100\}$. The same considerations can be applied when considering the time to incumbent with the exception of the number of customers for which the trend is not monotonic for the Smart-Path and Smart-Two-ways-path versions of the matheuristic. Moreover, looking at Table 8, we note that the time to incumbent is much lower than the total computational time. Thus, the computing time can be reduced without affecting the solution quality.

Table 8
Average time to incumbent (seconds).

	# instances	Smart-Chain	Smart-Path	Smart-Two-ways-path
$n < 28$	84	29	167	233
$28 \leq n < 48$	84	37	8	7
$48 \leq n \leq 98$	96	310	252	177
$\omega = 1/4$	88	56	15	19
$\omega = 1/2$	88	31	131	24
$\omega = 3/4$	88	322	290	400
$p_i = 1$	132	189	241	212
$p_i \in \{1, \ldots, 100\}$	132	84	50	83
$\pi_i = 0.5$	132	175	177	171
$\pi_i \in [0.25; 0.75]$	132	98	114	124
Total	**264**	**136**	**145**	**148**

8. Quality of the stochastic solution

The main difference between the POP and the classical OP is given by the uncertainty related to customers requests, i.e., a probability is associated with the fact that a customer will actually place a request. This motivates a different way of modeling and solving the problem as presented in the previous sections. An alternative way of dealing with this uncertainty could be to treat the problem as a deterministic one.

In this section, we aim at showing what is the value of developing a solution approach that explicitly takes into account the stochastic nature of the POP. This corresponds to the 'value of the stochastic solution' as proposed in [6]. To this aim, we compare the solution obtained through the exact algorithm presented in Section 5 with the solution obtained when a deterministic version of the problem is considered, which we call D-POP. In particular, the D-POP is the problem obtained by setting the profit associated with each customer $i \in V'$ to $p_i\pi_i$ and, afterwards, by setting all probabilities π_i to 1. This way, we obtain a routing problem with profits where the only difference with the standard OP is that the objective function maximizes the difference between the collected profit and the traveling cost (in the classical OP, the objective function consists in the maximization of the profit collected only). In order to compare the value of the solutions obtained by the two approaches, we then calculate the expected cost of the solution of D-POP as in (4).

To perform the analysis, we constructed a set of instances aimed at reflecting the real problem we described in the Introduction, where a company has to satisfy requests from different kinds

Table 9
Customer composition settings.

	Setting				
Subset	I	II	III	IV	V
V_M	0%	10%	20%	30%	40%
V_G	0%	10%	20%	30%	40%
V_R	0%	5%	15%	20%	30%

Table 10
Percentage gaps between D-POP solutions and POP solutions.

	Setting					
$\widehat{\pi}$	I	II	III	IV	V	All
0.05	–	207%	10%	6%	4%	45%
0.10	–	86%	11%	7%	4%	22%
0.20	100%	34%	10%	5%	2%	30%
0.30	84%	30%	6%	2%	1%	24%
0.40	1%	2%	2%	1%	0%	1%
0.50	0%	1%	1%	0%	0%	1%
All	31%	60%	7%	4%	2%	21%

of customers. In particular, we consider a class of 'mandatory' customers, i.e., customers which has a long-term contract with the company and thus has to be served in case they place a request. Let $V_M \subseteq V'$ be the set of mandatory customers. Notice that mandatory customers have to be inserted in the tour even if it is not sure that they will place their request. In order to include mandatory customers in our model, we just add the following constraints:

$$y_i = 1 \qquad (i \in V_M).$$

We define a second set $V_G \subseteq V'$ of customers, which we call 'guaranteed', with probability $\pi_i = 1$, i.e., they will surely place a request. However, contrary to what happens for the mandatory customers, the company may decide not to serve a customer in V_G. Finally, we define the set of 'spot' customers as $V_S = V' \setminus (V_M \cup V_G)$, i.e., customers which are neither mandatory nor guaranteed. In order to model the case where there are mandatory customers which will certainly place an order, we consider the set $V_R = V_M \cap V_G$, i.e., V_R contains the customers which will surely place an order and must be served. Table 9 describes five settings for percentages of customers included in the different subsets. All customers in $V' \setminus V_G$ have the same probability $\widehat{\pi}$ and we generated instances with the following values of $\widehat{\pi}$: 0.05, 0.10, 0.20, 0.30, 0.40, 0.50. Tests are made on TSP benchmark instance 'berlin52', which has 51 customers. The value of $D_{\max}$ is set to $(TSP(V_M) + TSP(V'))/2$, where $TSP(L)$ is the value of the optimal TSP solution over the set $L \cup \{0, n+1\}$. The value of the prize p_i is set to t_{0i} (which is equal to $t_{i,n+1}$) for all $i \in V'$.

The POP is solved to optimality through the `Two-ways-path` algorithm. The D-POP is solved through the same algorithm by setting the value of the customers prizes to $p_i^D = p_i \pi_i$ and $\pi_i^D = 1$ for all $i \in V'$. The maximum computational time is set to 5 h.

In Table 10, we compare the value z(D-POP) of the optimal solution of the D-POP, where the expected cost is calculated as in (4), with the value z(POP) of the optimal solution of the POP, when available, or the best solution found.

We note that the differences are quite substantial, especially for small values of $\widehat{\pi}$ and of $|V_M|$. The dashes in the first column of Table 10 are due to the fact that both solutions establish not to serve any customer. We also note that the difference goes up to 207%. This analysis shows that there is a value in taking into account the stochastic information in the problem formulation as done in the POP.

We now focus the analysis on the expected travel time of the solutions found by the POP. The aim is to analyze how much this value differs with respect to the maximum allowed route duration, i.e., $D_{\max}$. Table 11 reports the percentage gap between the expected travel time of the solution, calculated as in (4), and the value of $D_{\max}$. The table has the same format as Table 10.

Table 11
Percentage gaps between expected times and $D_{\max}$.

	Setting					
$\widehat{\pi}$	I	II	III	IV	V	All
0.05	–	70%	32%	20%	16%	48%
0.10	–	64%	29%	18%	15%	45%
0.20	45%	35%	25%	15%	12%	27%
0.30	35%	25%	22%	11%	9%	20%
0.40	28%	18%	18%	10%	8%	17%
0.50	22%	14%	15%	7%	6%	13%
All	55%	38%	24%	14%	11%	28%

Again, the two dashes in the first column are due to the fact that the optimal solution is the one where no customer is served. We notice that the gap tends to decrease when the cardinality of V_M increases and when $\widehat{\pi}$ increases. The minimum gap is 6%, meaning that the expected time is almost equal to $D_{\max}$. Note that constraint (9) guarantees that the solution is such that $D_{\max}$ is never exceeded.

In addition, to gain more insights on how much the actual travel time obtained a posteriori, i.e., once customers reveal their information, differs from $D_{\max}$, we took the solutions obtained when solving the POP and generated 1 million of realizations for each of them, on the basis of the customers selected in the a priori route and their probability. The results are shown in Fig. 2, where each plot corresponds to a given value of $\widehat{\pi}$. Travel time is represented on the horizontal axis (in thousands). On each plot, two vertical lines are represented: the total duration $T_{\max}$ of the a priori path and the value of $D_{\max}$.

As expected, the distributions are strictly related to the value of $\widehat{\pi}$. In particular, they are unbalanced on the left when $\widehat{\pi}$ is low, i.e., the actual duration differs quite substantially from $D_{\max}$, while they are unbalanced on the right when $\widehat{\pi} = 0.5$. This behavior is due to the fact that, when $\widehat{\pi}$ is small, the probability that customers visited in the a priori route effectively appear is low, thus resulting in an effective route duration which is much shorter than $T_{\max}$. The contrary happens when $\widehat{\pi}$ is large.

This analysis allows us to make the following considerations. The value of $D_{\max}$ should represent the maximum route duration. However, this value can be adapted to the instance at hand, i.e., if the customers have low probability, the a priori tour can be generated by relaxing the value of $D_{\max}$ and fixing it to a higher value. We expect the probability that the real value of $D_{\max}$ will be actually violated to be very low.

We may even use the POP to approximate an explicit chance constraint on the trip duration. Assume we wish to fix to α the probability that the trip duration $D^\star_{\max}$ will be exceeded. We may design a binary search on the $D_{\max}$ value in our POP model in order to fulfil approximately this constraint. The basic idea would be the following. We start from an interval of possible values for $D_{\max}$ of the form $[D^\star_{\max}, k \cdot D^\star_{\max}]$, for some $k > 1$. We fix $D_{\max}$ as the middle point and solve the corresponding POP, obtaining a certain solution $(x^\star, y^\star)$. We then generate an appropriate number of realizations of customer subsets, obtaining an estimate of the actual tour duration. If, according to this estimate, the probability to exceed $D^\star_{\max}$ is larger than α, we repeat the process on $[D^\star_{\max}, D_{\max}]$. Otherwise, we repeat the process on $[D_{\max}, k \cdot D^\star_{\max}]$.

The evaluation of such an approach is beyond the scope of this paper. Here, we make the following observation. Since chance constraints are hard to deal with and since, as shown in this paper, the POP is in itself a very hard problem, a model avoiding the ex-

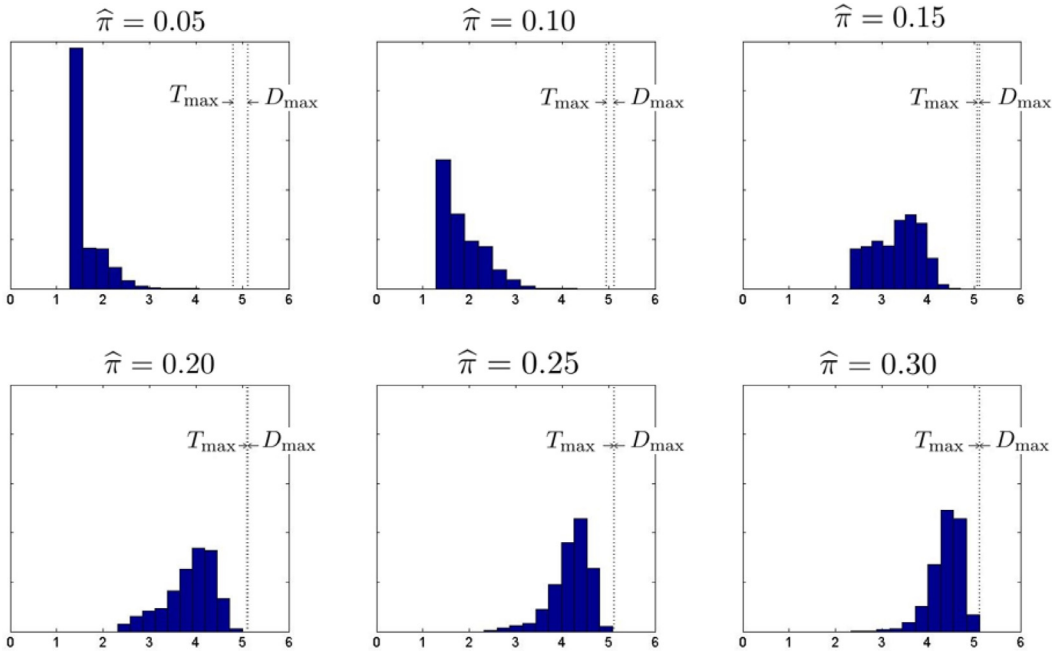

Fig. 2. Distribution of the actual solution travel time of 6 instances with a sample of 1 million realizations.

plicit inclusion of a chance constraint, as the one proposed in this paper, may be appropriate to maintain problem tractability.

9. Conclusions

We have defined the probabilistic orienteering problem (POP) as a stochastic variant of the orienteering problem (OP) where the availability of the customers for service is known only probabilistically and a recourse action is allowed to skip the absent customers when their availability is revealed. The objective is to maximise the difference between the expected prize (collected at visited customers) and the expected cost (proportional to the expected travel time).

We formulate the POP as a stochastic MILP and propose a branch-and-cut method for its solution. We also suggest heuristic strategies (matheuristics) to reduce the search space of the exact algorithm in order to get high quality solutions in a relatively short time.

We test the algorithms on a set of 264 instances. About 74% of them are solved to optimality by the branch-and-cut algorithms we proposed within 2 h. On average, the matheuristics found solutions with guaranteed error of less than 1% in a few minutes.

Finally, we perform an extensive analysis on a subset of instances to shown the value of modeling explicitly the stochastic information as done in the POP. To this aim, we compare POP solutions with solutions obtained when solving a deterministic version of the problem. The results show that there is value in modeling the stochastic information as the two solutions may be substantially different. Moreover, we also analyze the distribution of the actual solution duration on a subset of instances and show that, when the customer probabilities are high, it is strictly close to the value of the maximum route duration, while the opposite happens when the probabilities are low. Thus, in the latter case, one may relax the value of the maximum route duration to obtain a solution whose actual duration is close to the maximum route duration.

Future work may take two directions. The first direction concerns a different formulation of the problem, where the recourse action is modelled through a chance constraint on the time required by the selected path when some of the customers may be absent. A comparison of the results of this latter model with a POP with a relaxed duration constraint would be of interest. The second direction consists in the inclusion of the POP in a more accurate, dynamic approach to the real application we have described in the Introduction.

Acknowledgments

The authors wish to thank the anonymous referees whose valuable comments helped to improve the quality of the paper.

References

[1] Allahviranloo M, Chow JYJ, Recker WW. Selective vehicle routing problems under uncertainty without recourse. Transp Res Part E 2014;62(0):68–88.
[2] Archetti C, Speranza MG. A survey on matheuristics for routing problems. EURO J Comput Optim 2014;2:223–46.
[3] Archetti C, Speranza MG, Vigo D. Vehicle routing problems with profits. In: Toth P, Vigo D, editors. Vehicle routing: problems, methods, and applications, second edition, MOS-SIAM series on optimization, 18. Philadelphia: SIAM; 2014. p. 273–98.
[4] Bertsimas DJ, Jaillet P, Odoni AR. A priori optimization. Oper Res 1990;38(6):1019–33.
[5] Bertsimas DJ, Simchi-Levi D. A new generation of vehicle routing research: robust algorithms, addressing uncertainty. Oper Res 1996;44(2):286–304.
[6] Birge J. The value of the stochastic solution in stochastic linear programs with fixed recourse. Math Program 1982;24:314–25.
[7] Boschetti M, Maniezzo V, Roffilli M, Bolufé-Röhler A. Matheuristics: optimization, simulation and control. In: Blesa M, Blum C, Raidl G, Roli A, Sampels M, editors. Hybrid metaheuristics. Lecture Notes in Computer Science 6373. Springer Berlin Heidelberg; 2010. p. 171–7.
[8] Campbell AM, Gendreau M, Thomas BW. The orienteering problem with stochastic travel and service times. Ann Oper Res 2011;186(1):61–81.
[9] Campbell AM, Thomas BW. Challenges and advances in a priori routing. In: Golden B, Raghavan S, Wasil E, editors. The vehicle routing problem: latest advances and new challenges. Operations Research/Computer Science Interfaces, 43. Springer Science & Business Media; 2008. p. 123–42.
[10] Campbell AM, Thomas BW. Probabilistic traveling salesman problem with deadlines. Transp Sci 2008;42(1):1–21.
[11] Campbell AM, Thomas BW. Runtime reduction techniques for the probabilistic traveling salesman problem with deadlines. Comput Oper Res 2009;36(4):1231–48.
[12] Cordeau J-F, Laporte G, Savelsbergh MWP, Vigo D. Vehicle routing. In: Barnhart C, Laporte G, editors. Handbook in OR & MS: transportation, 14. Amsterdam, The Nederlands: Elsevier; 2007. p. 367–428.
[13] Dye S, Giffin J, Petty N. A subset-selection prize-collecting tsp with uncertain speed. In: Proceedings of the 45th annual conference of the ORSNZ; 2010.

[14] Evers L, Glorie K, van der Ster S, Barros AI, Monsuur H. A two-stage approach to the orienteering problem with stochastic weights. Comput Oper Res 2014;43(0):248–60.
[15] Feillet D, Dejax P, Gendreau M. Traveling salesman problems with profits. Transp Sci 2005;39(2):188–205.
[16] Fischetti M, González JJS, Toth P. A branch-and-cut algorithm for the symmetric generalized traveling salesman problem. Oper Res 1997;45(3):378–94.
[17] Fischetti M, González JJ, Toth P. Solving the orienteering problem through branch-and-cut. INFORMS J Comput 1998;10(2):133–48.
[18] Friedman M. The use of ranks to avoid the assumption of normality implicit in the analysis of variance. J Am Stat Assoc 1937;32(200):675–701.
[19] Gendreau M, Laporte G, Séguin R. Stochastic vehicle routing. Eur J Oper Res 1996;88(1):3–12.
[20] Golden BL, Levy L, Vohra R. The orienteering problem. Nav Res Logist (NRL) 1987;34(3):307–18.
[21] Heilporn G, Cordeau J-F, Laporte G. An integer -shaped algorithm for the dial-a-ride problem with stochastic customer delays. Discrete Appl Math 2011;159(9):883–95.
[22] İlhan T, Iravani SMR, Daskin MS. The orienteering problem with stochastic profits. IIE Trans 2008;40(4):406–21.
[23] Jaillet P. A priori solution of a traveling salesman problem in which a random subset of the customers are visited. Oper Res 1988;36(6):929–36.
[24] Jaillet P. Analysis of probabilistic combinatorial optimization problems in euclidean spaces. Math Oper Res 1993;18(1):51–70.
[25] Jaillet P, Odoni AR. The probabilistic vehicle routing problem. Vehicle routing: methods and studies North Holland, Amsterdam; 1988.
[26] Laporte G, Louveaux FV. The integer L-shaped method for stochastic integer programs with complete recourse. Oper Res Lett 1993;13(3):133–42.
[27] Laporte G, Louveaux FV, Mercure H. A priori optimization of the probabilistic traveling salesman problem. Oper Res 1994;42(3):543–9.
[28] Padberg M, Rinaldi G. A branch-and-cut algorithm for the resolution of large-scale symmetric traveling salesman problems. SIAM Rev 1991;33:60–100.
[29] Tang H, Miller-Hooks E. Algorithms for a stochastic selective travelling salesperson problem. J Oper Res Soc 2005;56(4):439–52.
[30] Tang H, Miller-Hooks E. Solving a generalized traveling salesperson problem with stochastic customers. Comput Oper Res 2007;34(7):1963–87.
[31] Tsiligirides T. Heuristic methods applied to orienteering. J Oper Res Soc 1984;35(9):797–809.
[32] Vansteenwegen P, Souffriau W, van Oudheusden D. The orienteering problem: a survey. Eur J Oper Res 2011;209(1):1–10.
[33] Vig V, Palekar US. On estimating the distribution of optimal traveling salesman tour lengths using heuristics. Eur J Oper Res 2008;186(1):111–19.
[34] Voccia SA, Campbell AM, Thomas BW. The probabilistic traveling salesman problem with time windows. EURO J Transp Logist 2013;2(1–2):89–107.
[35] Weyland D. On the computational complexity of the probabilistic traveling salesman problem with deadlines. Theor Comput Sci 2014;540–541(0):156–68.
[36] Weyland D, Montemanni R, Gambardella LM. Hardness results for the probabilistic traveling salesman problem with deadlines. In: Mahjoub AR, Markakis V, Milis I, Paschos VT, editors. Combinatorial optimization. Lecture Notes in Computer Science, 7422. Springer Berlin Heidelberg; 2012. p. 392–403.
[37] Weyland D, Montemanni R, Gambardella LM. Heuristics for the probabilistic traveling salesman problem with deadlines based on quasi-parallel monte carlo sampling. Comput Oper Res 2013;40(7):1661–70.
[38] Zhang S, Ohlmann JW, Thomas BW. A priori orienteering with time windows and stochastic wait times at customers. Eur J Oper Res 2014;239(1):70–9.